Host-Based Intrusion Detection Using Real-Time System Call Analysis with eBPF and an LSTM Model

Swaraj Joshi
School of Computer Science and Engineering
Vellore Institute of Technology, Vellore
Vellore-632014, Tamil Nadu, India
swaraj.joshi2024@vitstudent.ac.in

Abstract*—Signature-based security tools miss attacks they have never seen. A host-based intrusion detection system (HIDS) can instead learn how programs normally use the operating system and flag deviations. This project builds and evaluates such a system on a live Linux host. System calls are captured from the kernel with an eBPF program on the `raw_syscalls:sys_enter` tracepoint (or with ptrace/strace as a fallback that needs no root). Watched processes are followed through fork, exec and pid namespaces using cgroup v2. The detector was chosen in an offline machine learning study on the ADFA-LD and LID-DS datasets, where an LSTM next-call model (AUC 0.854) beat Isolation Forest, one-class SVM and an autoencoder. In the monitor this small LSTM (29,841 parameters, numpy inference, 28 MB monitor) predicts the next system call, and the per-process running mean of the prediction error is compared with two thresholds learned from normal runs only: the 99th percentile raises an alert, and 10% above the highest normal score stops or kills the process tree (SIGSTOP/SIGKILL, or `cgroup.freeze`/`cgroup.kill`). On 40 normal and 40 attack runs of a demo service (reconnaissance, privilege persistence, reverse shell, data exfiltration) the host-trained model reaches AUC 1.000 and F1 0.988 with one false alert, and kills 40 of 40 attacks with no normal run killed. With eBPF the monitor adds no measurable slowdown to ordinary programs, against ×7.7–×10 for strace. A model trained on the public ADFA-LD dataset fails on the same host (AUC 0.378), showing that the model must be trained on the host and capture method it protects.

Keywords—host-based intrusion detection, system calls, eBPF, cgroups, LSTM, anomaly detection, Linux kernel

I. Introduction

Every action a program takes that matters for security, such as opening a file, starting a process or connecting to the network, goes through the operating system kernel as a system call. When an attacker takes over a process, the attacker's code must also make system calls, so the sequence of calls a process makes is a precise and hard-to-avoid record of its behaviour. A HIDS that models this sequence can detect attacks for which no signature exists, because it looks for behaviour that is unusual for that program rather than for known malicious patterns [1], [2].

A. Problem Statement

Most published work on system-call anomaly detection is offline: models are trained and tested on recorded datasets such as ADFA-LD [3], [4], and reported results are detection scores on those files. This leaves open the questions an operating system raises in practice: how to capture every call of a process tree from the kernel without slowing it down, how to follow processes correctly across fork, exec, threads and pid namespaces, how to decide on a threshold without labelled attacks, how to respond automatically without killing healthy programs, and whether a model trained on one system works on another. This project addresses these questions by building a working real-time HIDS for Linux and measuring it on a live host.

B. Contributions

- A live monitor with two capture backends: an in-kernel eBPF tracepoint program filtered by cgroup id, and a ptrace (strace) backend that needs no root.
- Correct process tracking with cgroup v2, which is the same in every pid namespace, and an automatic response that suspends or kills the whole watched tree.
- A machine learning study comparing Isolation Forest, one-class SVM, an autoencoder and an LSTM on ADFA-LD and LID-DS, with an ablation of the LSTM's size, window, training data and scoring rule.
- An LSTM next-call model with alert and kill thresholds calibrated from normal runs only.
- An evaluation with four real attack payloads, live kill experiments, an overhead benchmark of both backends, and a cross-system test against the ADFA-LD dataset.

All code, results and instructions are published on GitHub [5].

II. Literature Review

A. Host-Based Intrusion Detection

Satilmiş *et al.* [2] review recent HIDS research and divide intrusion detection systems by data source into network-based and host-based systems; they list open problems and future directions for HIDS. Rahul-Vigneswaran *et al.* [6] survey deep learning and neural network models for both network and host intrusion detection. Early anomaly-based HIDS already learned normal behaviour without labels: Sujatha *et al.* [1] describe the host's behaviour with a "behavior set" of several parameters and cluster it with a self-organizing map trained by unsupervised competitive learning, so that new and novel attacks can be detected.

B. System Call Sequence Models

Subba *et al.* [3] turn system call traces into n-gram frequency vectors, reduce their dimension and classify them with SVM, MLP, Naive Bayes and decision trees on ADFA-LD, aiming at a low computational cost for real-time use. Suratkar *et al.* [7] combine an LSTM that detects anomalies in system call traces, modelling normal behaviour so that zero-day attacks can be found, with multiple hidden Markov models, one per known attack, that predict the next calls during an attack. Chaudhari *et al.* [8] combine an LSTM with system-call-frequency anomaly detection for virtual machines in the cloud and report 97.2% accuracy at a 2.4% false positive rate on ADFA-LD.

C. Datasets

ADFA-LD, recorded on 32-bit Ubuntu Linux, is the benchmark for system-call-based anomaly detection; Khandelwal *et al.* [4] compare the many machine learning studies that use it. The newer Leipzig Intrusion Detection Data Set (LID-DS) was used by Park *et al.* [9], who convert samples to images and classify them with a Siamese CNN trained by few-shot learning, improving recall by about 6% over a plain CNN.

D. Unsupervised Detectors

Detectors that learn only from normal data can find attacks never seen in training. Isolation Forest isolates anomalies with random axis-parallel splits; Xu *et al.* [10] note that this linear partition misses hard anomalies and propose a deep isolation forest. A one-class SVM learns a boundary around normal samples; Yerima and Bashar [11] use one for novelty detection that needs no labelled attacks. Zavrak and İskefiyeli [12] train an autoencoder and a variational autoencoder on normal network flows and compare them with a one-class SVM using ROC curves and AUC. LSTM networks model long sequences by keeping a memory of all earlier inputs, and outperform plain RNNs on sequential data [13].

E. Tracing and Containers

Craun *et al.* [14] show that eBPF tracing programs, being attached system-wide, also slow down processes that are not being traced, and compare filtering before, inside and after the eBPF program. Santana *et al.* [15] trace microservices through operating system system-call monitoring to avoid instrumenting the code and to keep overhead low. Containers group and isolate processes and resources; Xavier *et al.* [16] measure how well this isolation holds for disk-intensive workloads (up to 38% interference). Srinivasan *et al.* [17] build a real-time IDS for Docker containers using the probability of system call n-grams. Attacks keep adapting: host-oriented mimicry attacks imitate normal host and network behaviour to evade IDSs [18], and sigreturn-oriented programming (SROP) builds exploits from very few gadgets using the signal-return mechanism [19].

F. Research Gap

Most of the learned models above are trained and evaluated on recorded datasets, and the tracing studies measure cost but do not detect intrusions. Little work joins the two on a live host and measures the operating-system side: capture cost, process tracking, threshold choice for automatic response, and whether models transfer across systems. This project targets that gap.

III. Objectives

- Capture the full system call sequence of a process tree in real time from the kernel, with a fallback that needs no root.
- Follow every process and thread of the tree, including across pid namespaces (containers, WSL2).
- Score behaviour with a model trained only on normal runs, at a fixed and known false alarm rate.
- Respond automatically (stop or kill) without killing healthy programs.
- Detect real attack payloads injected into a running service.
- Keep monitoring overhead low enough for ordinary programs, and measure it.
- Compare with existing detectors and with a model trained on a public dataset.

IV. System Architecture

Fig. 1 shows the design. The watched program runs inside its own cgroup. In kernel space, either an eBPF program attached to the `raw_syscalls:sys_enter` and `sched:sched_process_exit` tracepoints emits one event per system call of that cgroup, or ptrace stops the program at each call for strace. In user space a single Python process parses the events, keeps a sliding window of calls per kernel task (thread), scores each window with the LSTM model, and keeps a running mean per task. Crossing the alert threshold logs an alert with the most anomalous window; crossing the higher response threshold stops or kills the process tree.

A. Components

- **Capture** (`live/capture.py`, `live/ebpf/syscalls.bt`): starts the program, creates its cgroup, attaches the tracer and turns raw output into events (call, process created, exec, exit, lost events).
- **Model** (`live/model.py`): LSTM next-call predictor, window statistics and threshold calibration.
- **Monitor** (`live/monitor.py`): the real-time loop, alerts, response and logging to `alerts.jsonl`.
- **Training and evaluation** (`live/train.py`, `live/evaluate.py`), **benchmark** (`live/bench/`), and **demo service and attacks** (`live/demo/`).

V. Methodology and Algorithm

A. Workload and Attacks

The protected program is a small "report service": a bash script that copies, counts, hashes, sorts and compresses files, using about 23 processes per run. A seed varies its inputs. Four attacks are injected into the same service: **recon** (`whoami`, `id` and a SUID-file scan with `find`), **adduser** (writes a uid-0 account line into a copy of `/etc/passwd`), **revshell** (connects back to a local listener and runs the commands it receives) and **exfil** (archives configuration files with `tar` and sends them over TCP). Test runs use seeds never seen in training.

B. Data Collection and Preprocessing

For each backend, normal runs are recorded into three disjoint sets: training (300 runs for eBPF, 100 for strace), calibration (300 / 100) and test (40 normal and 40 attack runs, 10 per payload). Each process gives one trace of x86_64 system call numbers. Traces are cut into windows of $L = 20$ calls with a step of 10. The vocabulary (47 calls) is built from training data only; unseen calls map to an "unknown" token. `rt_sigreturn` is removed from every trace, because its position depends on when an asynchronous signal arrives rather than on what the program does (see Section VIII). To score the 32-bit ADFA-LD model on live data, x86_64 calls are translated using the kernel's own tables (e.g. `mmap`→`mmap2`, `getuid`→`getuid32`, socket calls→`socketcall`).

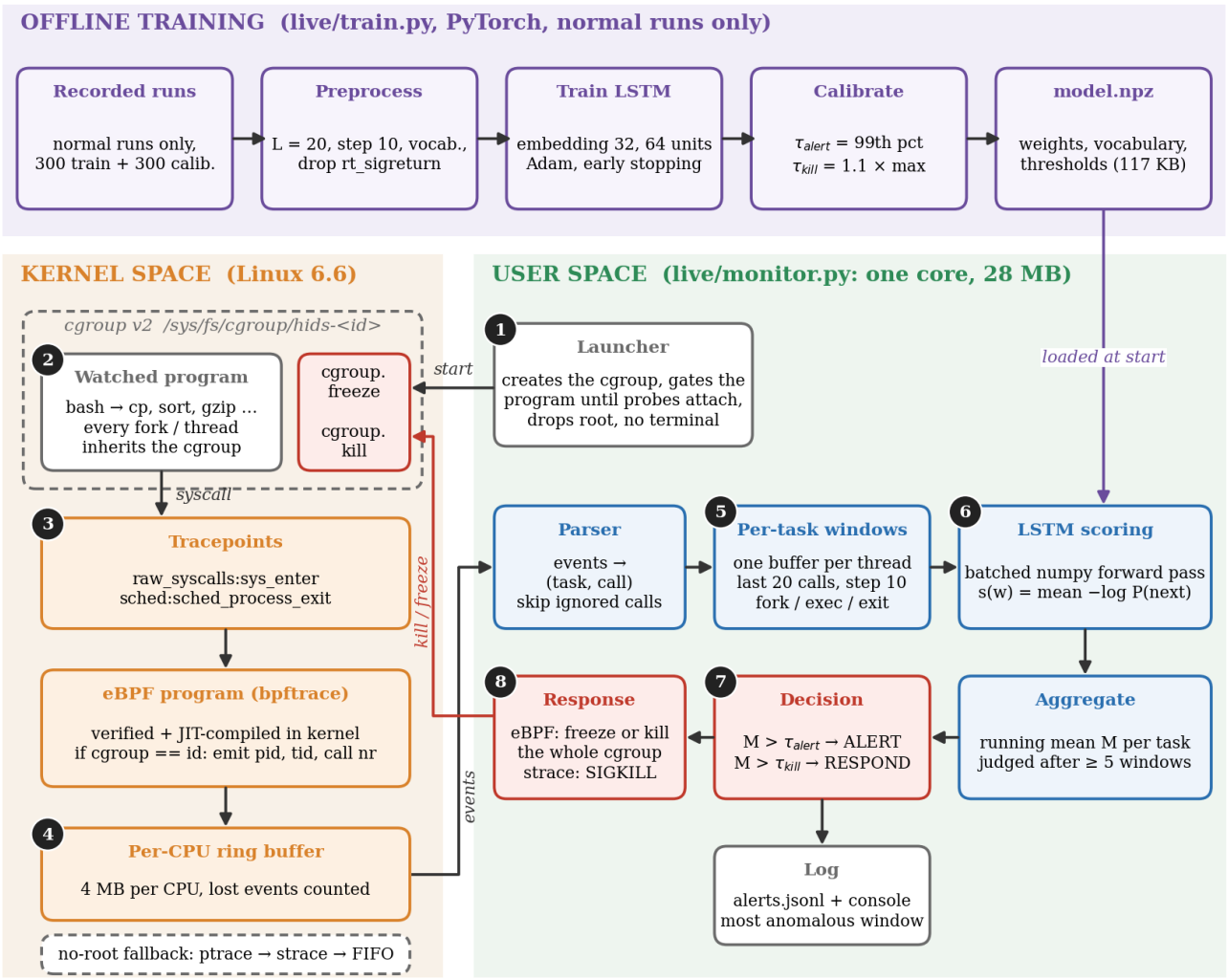


Fig. 1. Architecture of the live HIDS. Kernel-side capture (left) feeds the user-space detector (right); the response acts back on the watched cgroup.

C. Model

Each call c_t is embedded (32 dimensions) and fed to a single-layer LSTM with 64 hidden units, whose output predicts the next call:

$$h_t = \mathrm{LSTM}(E\, c_t,\, h_{t-1}) \tag{1}$$

$$P(c_{t+1} \mid c_{\le t}) = \mathrm{softmax}(W\, h_t + b) \tag{2}$$

The anomaly score of a window w is the mean negative log-likelihood of its calls:

$$s(w) = -(1/(L-1))\, \Sigma_{t=1..L-1} \log P(c_{t+1} \mid c_{\le t}) \tag{3}$$

The model is trained with cross-entropy and the Adam optimiser [20] in PyTorch (learning rate 2×10^{-3}, batch 256, early stopping after 3 epochs without improvement on held-out normal windows), then exported. The monitor runs inference in plain numpy, checked to match PyTorch within 4.5×10^{-7}.

D. Decision Rule and Thresholds

A single unusual window is too noisy to act on, so each task k keeps the running mean of its first n window scores, and is judged once it has at least 5 windows (or at exit if shorter):

$$M_k(n) = (1/n)\, \Sigma_{j=1..n}\, s(w_{k,j}) \tag{4}$$

$$S(\mathrm{run}) = \max_{k,\, n\ge5} M_k(n) \tag{5}$$

Both thresholds come from the calibration runs, which contain no attacks:

$$\tau_{\mathrm{alert}} = Q_{0.99}\{S(r)\}, \quad \tau_{\mathrm{kill}} = 1.1 \times \max\{S(r)\} \tag{6}$$

Using the same statistic for calibration and detection means the 99th percentile really gives about 1% false alerts. Killing a healthy program costs more than an alert, so the response needs a higher bar.

E. Detection Algorithm

1) Create cgroup G; start the program held at a gate inside G; attach the tracer filtered on G; release the gate.

2) For each event (*task*, *call*): skip ignored calls; append *call* to the task's buffer.

3) Every 10 new calls, form a window of the last 20; batch windows and compute $s(w)$ with (3).

4) Update M_k with (4). If $n \ge 5$ and $M_k > \tau_{\mathrm{alert}}$: log an alert once, with the most anomalous window.

5) If $M_k > \tau_{\mathrm{kill}}$ and a response is enabled: freeze or kill G (eBPF) or send SIGSTOP/SIGKILL (strace).

6) On task exit: compute its final verdict. When the tree ends: print a summary; exit code 3 if any alert.

VI. Machine Learning Model Selection

The detector used by the live monitor was chosen in an offline study that compared four unsupervised models on public datasets (`phases/`, `phase6b_improve.py`). All of them learn from normal behaviour only, so they can flag attacks never seen before.

A. Datasets and Protocol

ADFA-LD [3], [4] contains 833 normal training traces, 4,372 normal validation traces and 746 attack traces (mean length 370, 485 and 425 calls) from a 32-bit Ubuntu 11.04 server attacked with password brute force, a web shell, a Java exploit, a Meterpreter payload and new users with root rights. LID-DS 2021 [9] (scenario Bruteforce_CWE-307) was used as a second, more recent dataset. The protocol was kept strict: models and the vocabulary (150 calls) are built from normal training data only; the 746 attack traces and the same number of normal validation traces are split into a tuning half and a test half; every setting was chosen on the tuning half, and the test half was used once for the final result; each result is the mean of 5 random seeds.

B. Features and Detectors

Isolation Forest and one-class SVM need fixed-length vectors, so each window of 20 calls is turned into a 151-bin histogram of calls plus 256 hashed bigram counts (407 features, 96.7% zeros), optionally with 256 hashed trigrams (663 features). These features keep only short-range order. The four detectors were:

- **Isolation Forest** [10]: 100 random trees; anomalies are isolated in fewer splits.
- **One-class SVM** [11]: RBF kernel, $\nu = 0.05$; learns a boundary around normal feature vectors.
- **Autoencoder** [12]: fully connected 407→64→16→64→407 network (ReLU, Adam, early stopping); the score is the reconstruction error (MSE), which is high for patterns it never learned to compress.
- **LSTM next-call model** [7], [8], [13]: sees the raw call sequence in order and scores each window by (3).

The window score is turned into a trace score with a scoring rule (mean, maximum, or mean of the top 10% of windows), and the threshold is the 95th or 99th percentile of normal calibration scores.

C. Detector Comparison

TABLE I. Detectors on ADFA-LD (5 Seeds, 95th-Percentile Threshold)

Detector	AUC	Precision	Recall	F1	FAR
Isolation Forest [10]	0.668	0.804	0.207	0.329	0.050
One-class SVM [11]	0.479	0.635	0.082	0.146	0.048
Autoencoder	0.752	0.584	0.070	0.125	0.050
LSTM next-call	**0.828**	0.865	0.276	**0.417**	0.042

Mean of 5 seeds; standard deviations ≤ 0.034 (AUC). Time per window: 0.012–0.135 ms for all four.

The LSTM is best on every measure that does not depend on the threshold, and Fig. 2 shows its ROC curve [21] above the others at the low false alarm rates that matter for a HIDS. The one-class SVM is no better than chance on these sparse features. Adding trigrams raised Isolation Forest only from AUC 0.612 to 0.636, which shows that the order information the LSTM uses is what separates attacks from normal traces.

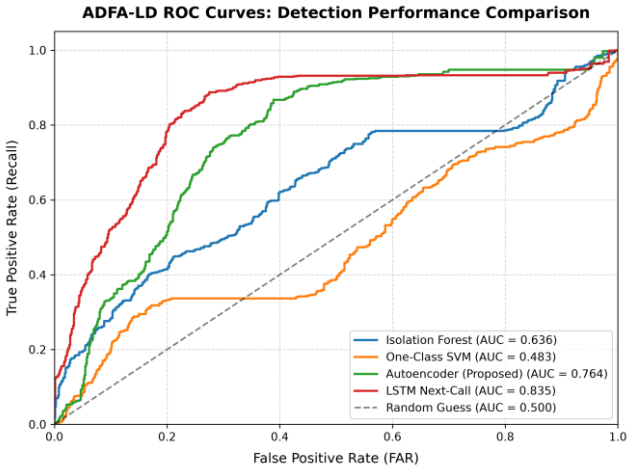


Fig. 2. ROC curves of the four detectors on the ADFA-LD test windows (one seed, trigram features for the baselines).

D. Tuning the LSTM

Fig. 3 shows the ablation on the tuning half. A larger model (128 units), a second layer or longer windows (50 or 100 calls) did not help (AUC 0.811–0.825). Moving 2,000 normal validation traces into training raised AUC from 0.822 to 0.859: *more normal data mattered more than model size*. An ensemble of 3 models added only 0.002 at 2.3 times the training time, so a single small model was kept. Of the scoring rules, the mean of all windows was best (AUC 0.859, against 0.804 for the top 10% and 0.771 for the maximum), which is why the live monitor uses a running mean.

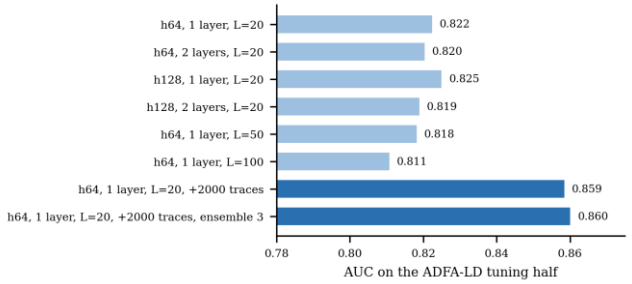


Fig. 3. LSTM ablation on the ADFA-LD tuning half (mean-window scoring); dark bars use 2,000 extra normal training traces.

TABLE II. Final LSTM on the ADFA-LD Test Half (5 Seeds)

Threshold	Precision	Recall	F1	FAR	AUC
99th pct (normal)	0.923	0.097	0.175	0.008	0.854
95th pct (normal)	0.875	0.377	**0.527**	0.054	0.854
Tuned (uses attacks)	0.751	0.923	0.828	0.306	0.854

LID-DS Bruteforce_CWE-307 (3 seeds): AUC 1.000, F1 0.812 at the 99th percentile, FAR 0.084.

On ADFA-LD the final model reaches AUC 0.854 and F1 0.527 at a 5.4% false alarm rate using thresholds from normal data only; a threshold tuned with attack labels gives F1 0.828 but flags 30.6% of normal traces. The goal of F1 0.90 was not met on ADFA-LD, whose attacks are hidden inside long, varied normal sessions. On LID-DS every attack is found (recall 1.0). The LSTM was taken into the live monitor because it had the best AUC, it uses call order, and it is small:

29,841 parameters, 117 KB, and 0.03 ms per window on one core.

VII. Implementation

A. Technology Stack

TABLE III. Technology Stack

Layer	Technology
OS / platform	Linux kernel 6.6 (WSL2), Ubuntu 22.04, x86_64
Kernel capture	eBPF via bpftrace 0.14; ptrace via strace
Process control	cgroup v2 (`cgroup.freeze`, `cgroup.kill`), signals
Language	Python 3.10, bash, C (benchmark)
Model training	PyTorch (CPU)
Inference / data	numpy, scikit-learn (baselines), psutil
Plots / results	matplotlib, CSV, JSON Lines logs
Version control	Git, GitHub [5]

B. Kernel-Side Capture with eBPF

The bpftrace script has two probes, each filtered by `cgroup == $1`. The entry probe prints process id, thread id, call number and command name; the exit probe prints the thread id. Filtering on the cgroup id instead of a pid is essential: eBPF reports kernel-global pids, while processes in a container or a WSL2 distro see namespace-local pids, so following "pid 1234" silently matches nothing. The cgroup id (the inode of the cgroup directory) is the same everywhere, and every child inherits the cgroup inside the kernel, so no fork can be missed. This is the "in-eBPF" filtering studied by Craun *et al.* [14]: other processes still run the probe, but only for a cheap comparison, and the cgroup is the same grouping that containers use for isolation [16].

To avoid losing the first calls, the program is started behind a small bash *gate* that waits on a pipe inside the cgroup until bpftrace prints that its probes are attached, then `exec`s the real program (keeping its `argv[0]`). The gate's own calls before that `execve` are dropped. The per-CPU ring buffer was raised to 4 MB (`BPFTRACE_PERF_RB_PAGES=1024`) after the default size lost 33,616 of 50,582 events in a burst; losses are always counted and reported.

C. Following Processes and Responding

Windows are kept per kernel task, so threads do not mix. Names are updated on every successful `execve`. With strace, new children are read from the return value of `clone`, `fork` and `vfork`. The response uses kernel mechanisms that the target cannot block: SIGSTOP/SIGKILL for strace, and writing to the cgroup's `cgroup.freeze` or `cgroup.kill` file for eBPF, the same interfaces container runtimes use (Linux ≥ 5.14).

D. Controlling the Environment

The trace depends on how a program is started, not only on what it does. A terminal adds job-control calls and bash's `checkwinsize` queries the window size after every command; `sudo` replaces PATH; running as root adds permission checks. Each of these raised false alarms. The monitor therefore starts the program like a service (own session, stdin from `/dev/null`, output relayed through a pipe), keeps the user's PATH, and drops from root to the invoking user. While preparing this report the effect was reproduced: the same normal run started from a non-login shell, with a different PATH, was flagged and killed, while from a login shell it raised no alert.

E. Testing and Correctness

Offline unit tests (`live/tests/test_live.py`) check the strace and bpftrace parsers (including interrupted and resumed calls), the x86_64→i386 mapping and the running-mean statistic; all pass. Training checks numpy against PyTorch scores on 500 windows (maximum difference 4.47×10^{-7}). The evaluation scripts recompute every number in this report from the stored recordings.

F. Demonstration of Working Output

Figs. 4–6 show the monitor's real output on the host (strace backend, `--respond kill`). In a normal run (Fig. 4) all 15 processes stay below the alert threshold. In the reverse-shell attack (Fig. 5) bash is killed at the `socket connect … dup2` sequence that wires the shell to the network, and the spawned shell (score 0.980) is killed too. In the exfiltration attack (Fig. 6) `tar`, bash and `cat` are killed while the data is collected.

```
$ python3 live/monitor.py --model live/models/host.npz --respond kill --
      bash live/demo/service.sh 7
[hids] model live/models/host.npz: numbering x86_64, window 20/10, alert
      threshold live_p99 = 0.170
[hids] response kill only above live_max + 10% = 0.199

   tid     pid program            calls windows   score  verdict
 89979   89979 rm                   169      15   0.152  normal
 89967   89967 ls                   223      21   0.138  normal
 89965   89965 bash                 695      68   0.137  normal
 89966   89966 mktemp               173      16   0.134  normal
 89976   89976 sort                 188      17   0.099  normal
 89975   89975 grep                 181      17   0.094  normal
 89971   89971 grep                 181      17   0.094  normal
 89968   89968 cp                   210      20   0.083  normal
 89972   89972 cp                   210      20   0.083  normal
 89977   89977 gzip                  83       7   0.076  normal
 89978   89978 date                 151      14   0.057  normal
 89970   89970 sha256sum            151      14   0.051  normal
 89974   89974 sha256sum            151      14   0.051  normal
 89969   89969 wc                   149      13   0.045  normal
 89973   89973 wc                   149      13   0.045  normal

[hids] 3078 system calls from 15 tasks in 0.29 s (10774/s)
[hids] monitor CPU 0.08 s (28.0% of one core), peak RSS 28.3 MB
[exit code 0]
```

Fig. 4. Normal run of the report service: 3,078 calls from 15 processes, 0 alerts, exit code 0.

```
$ ATTACK=revshell python3 live/monitor.py --model live/models/host.npz
      --respond kill -- bash live/demo/service.sh 7
[hids] model live/models/host.npz: numbering x86_64, window 20/10, alert
      threshold live_p99 = 0.170
[hids] response kill only above live_max + 10% = 0.199

[ALERT 21:22:47] bash (pid 90000, tid 90000) mean score 0.238 > 0.170
      after 51 windows / 569 calls (running)
          most anomalous window: rt_sigprocmask rt_sigaction wait4
      rt_sigaction ioctl rt_sigprocmask wait4 rt_sigprocmask
      rt_sigprocmask read socket connect fcntl fcntl fcntl fcntl dup2
      fcntl ioctl lseek
[RESPONSE 21:22:47] bash (pid 90000) score 0.238 > 0.199: sent SIGKILL
      to pid 90000

[ALERT 21:22:47] bash (pid 90012, tid 90012) mean score 0.980 > 0.170
      after 5 windows / 110 calls (running)
          most anomalous window: rt_sigaction rt_sigaction rt_sigaction
      rt_sigaction rt_sigaction rt_sigaction dup2 fcntl dup2 fcntl
      execve brk arch_prctl mmap access openat newfstatat mmap close
      openat
[RESPONSE 21:22:47] bash (pid 90012) score 0.980 > 0.199: sent SIGKILL
      to pid 90012

   tid     pid program            calls windows   score  verdict
 90012   90012 bash                 163      15   0.440  ANOMALOUS
 90000   90000 bash                 569      55   0.372  ANOMALOUS

[hids] 2520 system calls from 12 tasks in 0.25 s (10280/s)
[hids] monitor CPU 0.08 s (32.6% of one core), peak RSS 28.2 MB
[exit code 3]
```

Fig. 5. Reverse-shell attack: both bash processes involved are flagged and killed with SIGKILL.

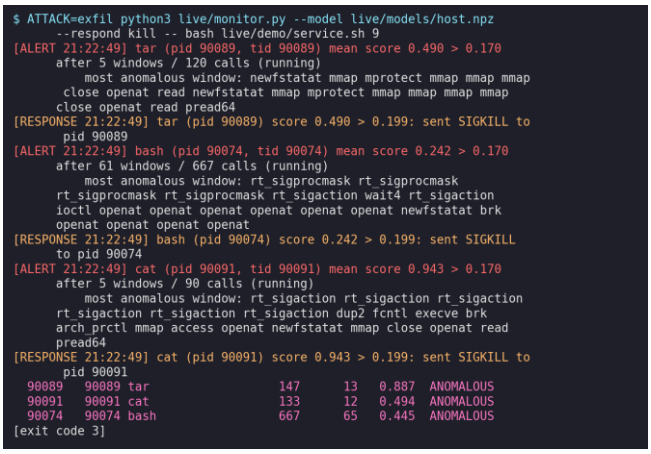


Fig. 6. Exfiltration attack: tar, bash and cat are flagged and killed during the payload.

VIII. Results and Performance Evaluation

A. Experimental Setup and Metrics

All experiments ran on WSL2 (Ubuntu 22.04, kernel 6.6.87, x86_64). A run counts as flagged if any of its tasks crosses the threshold. We report AUC [21] of the run score, precision, recall, F1 and the false alarm rate (FAR, share of normal runs flagged) on the 40 normal and 40 attack test runs.

B. Detection Results

TABLE IV. Run-Level Detection at the Alert Threshold

Trained on	Tested on	AUC	Prec.	Recall	F1	FAR
Host, eBPF	eBPF	1.000	0.976	1.000	**0.988**	**0.025**
Host, strace	strace	1.000	0.976	1.000	**0.988**	**0.025**
Host, strace	eBPF	0.994	0.500	1.000	0.667	1.000
Host, eBPF	strace	1.000	0.500	1.000	0.667	1.000
ADFA-LD	strace	0.378	0.500	1.000	0.667	1.000

40 normal and 40 attack test runs per row; 1 of 40 normal runs raised an alert in the matched rows.

When the model is trained with the same capture method it is used with, every attack is detected and only one normal run raises an alert, as expected from a 99th-percentile threshold. Fig. 7 shows why: all normal runs sit just at the alert threshold, while attacks score from 0.32 to over 3.

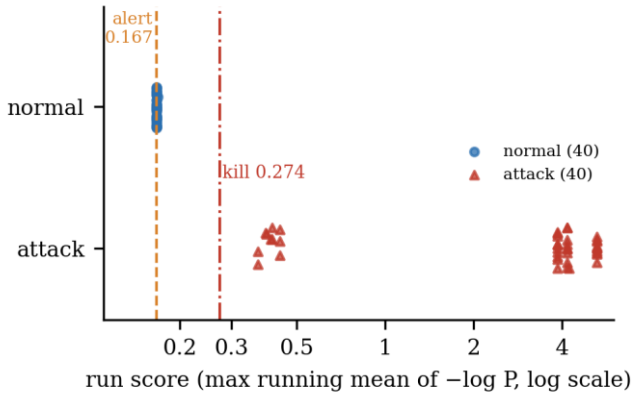


Fig. 7. Run scores of the eBPF-trained model on eBPF-captured test runs, with the alert and kill thresholds.

Two cross results matter for operating systems. First, *the observer effect*: ptrace stops the program at every call and so fixes the order of events such as a child's exit and bash's `wait4`; under eBPF the program runs at full speed and other orderings appear. A strace-trained model therefore flags every normal eBPF-captured run, although its AUC (0.994) shows attacks still stand out. Second, the ADFA-LD model scores worse than chance (AUC 0.378) and flags every normal run: fourteen years of kernel and glibc changes alter normal behaviour more than the attacks do. A model must be trained on the host, environment and capture method it protects.

C. Alert versus Response

TABLE V. Alerts and Kills on the Test Runs (Offline)

Trained on / tested on	Normal alerted	Normal killed	Attacks killed
eBPF / eBPF	1 / 40	**0 / 40**	**40 / 40**
strace / strace	1 / 40	**0 / 40**	**40 / 40**
strace / eBPF	40 / 40	22 / 40	40 / 40
eBPF / strace	40 / 40	0 / 40	40 / 40

Kill threshold τ_{kill}: 0.199 (strace model), 0.274 (eBPF model).

Live, with `--respond kill` and the final models, the strace backend raised no alert on 12 of 12 normal runs (6 seeds, with and without a terminal) and killed every attack during its payload: recon at `id`, adduser at bash after the passwd copy, the reverse shell at `connect … dup2`, exfil at `tar` and `cat`. With the eBPF backend, 5 of 5 normal runs on unseen seeds were not killed (one raised an alert only, at 0.198, between the two thresholds), and every attack was killed by killing its cgroup, with scores from 0.366 to 3.570.

D. Monitoring Overhead

TABLE VI. Workload Slowdown (Median of 5 Repeats)

Workload	strace + monitor	eBPF only	eBPF + monitor
Syscall storm, 50 k calls	×259	×6.5	**×6.4**
Report service ×10, 43 k calls	×10.1	×1.13	**×0.88**
gzip 32 MB, 1.7 k calls	×1.32	×1.06	**×1.04**

Baselines: 0.015 s, 0.361 s, 0.466 s. eBPF lost 0 events in every run.

Overhead grows with the *system call rate*, not with the work done (Fig. 8). ptrace stops the program twice per call and wakes the tracer, so a tight loop of calls runs about 200 times slower; under eBPF each call only runs a small in-kernel program (about 1.6 µs). Because that program is attached system-wide, untraced processes also pay for the cgroup check, the cost Craun *et al.* [14] set out to remove. For ordinary programs eBPF is within run-to-run noise: the unmonitored service itself varied from 0.24 to 0.42 s, so ×0.88 means no measurable cost, not a speed-up. Because the kernel buffers events, the program never waits for the detector. The monitor used 28 MB of memory and about 0.1 ms per scored window.

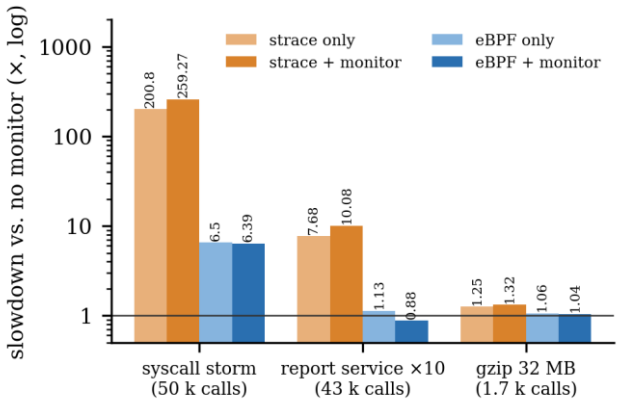


Fig. 8. Slowdown of three workloads under each monitoring mode (log scale; 1 = no monitor).

E. Comparison with Existing Methods

Section VI compared the LSTM with Isolation Forest, one-class SVM and an autoencoder on ADFA-LD under one protocol: the LSTM had the highest AUC (0.854 against at most 0.752). Published results on ADFA-LD are higher: Chaudhari *et al.* [8] report 97.2% accuracy at a 2.4% false positive rate, and the classifiers of Subba *et al.* [3] are trained with labelled attacks. These figures use other metrics, splits and labels, so they are not directly comparable [4]; with a threshold tuned on attack labels our model reaches F1 0.828, but at a 30.6% false alarm rate, which is unusable for automatic response. Unlike those offline studies, and unlike the container IDS of Srinivasan *et al.* [17], which was evaluated on a recorded dataset, this system runs on a live host, responds by itself and reports its own overhead. It needs no attack signatures and detected all four payloads, but it needs normal training runs for every protected program.

F. Comparison with Expected Outcomes

TABLE VII. OBJECTIVES AND OUTCOMES

Objective	Outcome
Real-time kernel capture	Met: eBPF and strace backends, 0 events lost
Correct process tracking	Met: cgroup v2, works across pid namespaces
Known false alarm rate	Met: 1/40 alerts at the 99th percentile
Safe automatic response	Met: 0/40 normal killed, 40/40 attacks killed
Detect real payloads	Met: all 4 attack types, live and offline
Low overhead	Met for eBPF (×0.88–×1.04); strace up to ×259
ML model selection	Met: LSTM best (AUC 0.854); F1 goal 0.90 not met
Transfer from public data	Not met: ADFA-LD model AUC 0.378

IX. LIMITATIONS

- The demo service is small and repetitive, so the results show that the pipeline works end to end, not accuracy on a busy general-purpose server.
- Three choices were made after seeing results: separate alert and kill thresholds, the 10% kill margin, and ignoring `rt_sigreturn`. A subtler attack might alert without being killed, and ignoring `rt_sigreturn` hides the call that SROP exploits abuse [19].
- About 1% of normal runs raise an alert, as a 99th-percentile threshold implies.
- The model also learns the environment (PATH, user, `argv[0]`), so programs must run as they did during training.
- Only call numbers are used, so mimicry attacks [18] remain possible.
- Whole-host eBPF mode can produce more events than a single Python consumer can score.

X. CONCLUSION AND FUTURE WORK

This project turned an offline system-call anomaly detector into a working real-time HIDS on Linux. eBPF capture filtered by cgroup follows a whole process tree across fork, exec and pid namespaces at no measurable cost for ordinary programs, while ptrace remains a slower fallback. An LSTM next-call model with thresholds learned from normal runs detected every attack payload, and the two-threshold policy killed every attack and no normal run on the test set and in live runs. The experiments also showed that operating-system details decide whether detection works: the capture method changes event ordering, the launch environment changes the trace, and a model trained on another system fails.

Future work: use call arguments and return values against mimicry attacks, move scoring into the kernel or a compiled consumer for whole-host monitoring, learn normal behaviour of real services such as web and database servers, and update the model online as programs change.

SOURCE CODE

The complete source code, recorded results and instructions are available at https://github.com/swarajjoshi10-ship-it/host-based-instrusion-detection [5].

REFERENCES

[1] P. K. Sujatha, A. Kannan, S. Ragunath, K. S. Bargavi, and S. Githanjali, "A behavior based approach to host-level intrusion detection using self-organizing maps," in *Proc. 1st Int. Conf. Emerging Trends in Engineering and Technology*, Nagpur, India, 2008, pp. 1267–1271, doi: 10.1109/ICETET.2008.190.

[2] H. Satilmiş, S. Akleylek, and Z. Y. Tok, "A systematic literature review on host-based intrusion detection systems," *IEEE Access*, vol. 12, pp. 27237–27266, 2024, doi: 10.1109/ACCESS.2024.3367004.

[3] B. Subba, S. Biswas, and S. Karmakar, "Host based intrusion detection system using frequency analysis of n-gram terms," in *Proc. IEEE Region 10 Conf. (TENCON)*, Penang, Malaysia, 2017, pp. 2006–2011, doi: 10.1109/TENCON.2017.8228190.

[4] P. Khandelwal, P. Likhar, and R. S. Yadav, "Machine learning methods leveraging ADFA-LD dataset for anomaly detection in Linux host systems," in *Proc. 2nd Int. Conf. Intelligent Technologies (CONIT)*, Hubli, India, 2022, pp. 1–8, doi: 10.1109/CONIT55038.2022.9848305.

[5] S. Joshi, "Host-based intrusion detection using system call analysis," GitHub repository, 2026. [Online]. Available: https://github.com/swarajjoshi10-ship-it/host-based-instrusion-detection

[6] K. Rahul-Vigneswaran, P. Poornachandran, and K. P. Soman, "A compendium on network and host based intrusion detection systems," in *ICDSMLA 2019*, A. Kumar, M. Paprzycki, and V. K. Gunjan, Eds. Singapore: Springer, 2020, pp. 23–30, doi: 10.1007/978-981-15-1420-3_3.

[7] S. Suratkar, F. Kazi, R. Gaikwad, A. Shete, R. Kabra, and S. Khirsagar, "Multi hidden Markov models for improved anomaly detection using system call analysis," in *Proc. IEEE Bombay Section Signature Conf. (IBSSC)*, Mumbai, India, 2019, pp. 1–6, doi: 10.1109/IBSSC47189.2019.8973098.

[8] A. Chaudhari, B. Gohil, and U. P. Rao, “A novel hybrid framework for cloud intrusion detection system using system call sequence analysis,” *Cluster Computing*, vol. 27, no. 3, pp. 3753–3769, 2024, doi: 10.1007/s10586-023-04162-z.

[9] D. Park, S. Kim, H. Kwon, D. Shin, and D. Shin, “Host-based intrusion detection model using Siamese network,” *IEEE Access*, vol. 9, pp. 76614–76623, 2021, doi: 10.1109/ACCESS.2021.3082160.

[10] H. Xu, G. Pang, Y. Wang, and Y. Wang, “Deep isolation forest for anomaly detection,” *IEEE Trans. Knowledge and Data Engineering*, vol. 35, no. 12, pp. 12591–12604, Dec. 2023, doi: 10.1109/TKDE.2023.3270293.

[11] S. Y. Yerima and A. Bashar, “Semi-supervised novelty detection with one class SVM for SMS spam detection,” in *Proc. 29th Int. Conf. Systems, Signals and Image Processing (IWSSIP)*, Sofia, Bulgaria, 2022, pp. 1–4, doi: 10.1109/IWSSIP55020.2022.9854496.

[12] S. Zavrak and M. İskefiyeli, “Anomaly-based intrusion detection from network flow features using variational autoencoder,” *IEEE Access*, vol. 8, pp. 108346–108358, 2020, doi: 10.1109/ACCESS.2020.3001350.

[13] B. B. Sahoo, R. Jha, A. Singh, and D. Kumar, “Long short-term memory (LSTM) recurrent neural network for low-flow hydrological time series forecasting,” *Acta Geophysica*, vol. 67, no. 5, pp. 1471–1481, 2019, doi: 10.1007/s11600-019-00330-1.

[14] M. Craun, K. Hussain, U. Gautam, Z. Ji, T. Rao, and D. Williams, “Eliminating eBPF tracing overhead on untraced processes,” in *Proc. ACM SIGCOMM 2024 Workshop on eBPF and Kernel Extensions*, Sydney, Australia, 2024, pp. 16–22, doi: 10.1145/3672197.3673431.

[15] M. Santana, A. Sampaio, M. Andrade, and N. S. Rosa, “Transparent tracing of microservice-based applications,” in *Proc. 34th ACM/SIGAPP Symp. Applied Computing (SAC)*, Limassol, Cyprus, 2019, pp. 1252–1259, doi: 10.1145/3297280.3297403.

[16] M. G. Xavier, I. C. De Oliveira, F. D. Rossi, R. D. Dos Passos, K. J. Matteussi, and C. A. F. De Rose, “A performance isolation analysis of disk-intensive workloads on container-based clouds,” in *Proc. 23rd Euromicro Int. Conf. Parallel, Distributed, and Network-Based Processing (PDP)*, Turku, Finland, 2015, pp. 253–260, doi: 10.1109/PDP.2015.67.

[17] S. Srinivasan, A. Kumar, M. Mahajan, D. Sitaram, and S. Gupta, “Probabilistic real-time intrusion detection system for Docker containers,” in *Security in Computing and Communications*, S. M. Thampi *et al.*, Eds. Singapore: Springer, 2019, pp. 336–347, doi: 10.1007/978-981-13-5826-5_26.

[18] S. Sanyal, M. Khatua, and P. Chattopadhyay, “CIPHER: A collaborative IDS placement and scheduling framework for host-oriented mimicry attack detection in IoT networks,” *IEEE Internet of Things J.*, vol. 13, no. 14, pp. 31880–31888, Jul. 2026, doi: 10.1109/JIOT.2026.3689273.

[19] Z. Mithra and Vipin P., “Evaluating the theoretical feasibility of an SROP attack against Oxymoron,” in *Proc. Int. Conf. Advances in Computing, Communications and Informatics (ICACCI)*, Kochi, India, 2015, pp. 1872–1876, doi: 10.1109/ICACCI.2015.7275891.

[20] M. Reyad, A. M. Sarhan, and M. Arafa, “A modified Adam algorithm for deep neural network optimization,” *Neural Computing and Applications*, vol. 35, no. 23, pp. 17095–17112, 2023, doi: 10.1007/s00521-023-08568-z.

[21] A. M. Carrington *et al.*, “Deep ROC analysis and AUC as balanced average accuracy, for improved classifier selection, audit and explanation,” *IEEE Trans. Pattern Analysis and Machine Intelligence*, vol. 45, no. 1, pp. 329–341, Jan. 2023, doi: 10.1109/TPAMI.2022.3145392.